

A Serverless Hybrid ETL Architecture for Real-Time E-Commerce Analytics

K. Bruhadesh Varma

Department of Computer Science
Amrita School of Computing
Amrita Vishwa Vidyapeetham
Amritapuri, Kollam, India

am.sc.u4aie23039@am.students.amrita.edu

R. Teja

Department of Computer Science
Amrita School of Computing
Amrita Vishwa Vidyapeetham
Amritapuri, Kollam, India

am.sc.u4aie23053@am.students.amrita.edu

C. Tanmay Srinivas

Department of Computer Science
Amrita School of Computing
Amrita Vishwa Vidyapeetham
Amritapuri, Kollam, India

am.sc.u4aie23059@am.students.amrita.edu

Guide: Dr. Divya Udayan J

Department of Computer Science
Amrita School of Computing
Amrita Vishwa Vidyapeetham
Amritapuri, Kollam, India

divyaudayanj@am.amrita.edu

Abstract—The paper describes a neat automated ETL pipeline from the cloud for e-commerce transactional data processing on Google Cloud Platform (GCP). The mechanism proposed for the planned architecture involves Google Cloud Storage for raw data ingestion, a serverless Cloud Run microservice to perform scalable data transformations with low latency, BigQuery as an optimized analytical store, and Looker Studio for real-time dashboard visualization. A custom-built ETL Control Dashboard hosted on Firebase enables secure triggering and execution monitoring of the pipeline, module-wise ETL operation, and real-time transactional record insertion via REST APIs. This system will demonstrate the performance levels of enterprise-grade automation, fault tolerance, and low operational overhead. This extended version is meant to give a more in-depth technical analysis along with context, literature review, and structure improvements and a detailed performance evaluation based on multiple sizes of datasets. Future architectural and functional improvements are outlined to place the pipeline in the context of contemporary cloud data engineering.

ETL Pipeline, Cloud Computing, Google Cloud Platform, BigQuery, Cloud Run, Looker Studio, E-Commerce Analytics.

I. INTRODUCTION

The present digital commerce portals are a utility that generates immense transaction data, including invoice numbers, product metadata with attributes, customers, quantity, and price information, along with time and geographical identifiers. This data represents significant business potential by allowing organizations to analyze revenue trends, study customer segments, optimize product performance, and find operational inefficiency. However, the raw data is usually inconsistent, noisy, unstructured, and thus, not directly analyzable. Hence, the quality of data and downstream analytics is ensured through solid ETLs.

Conventional ETLs rely on various means such as virtual machine or monolithic script or manual batch processes. All these approaches come with issues like limited scalability, high operational maintenance, increased processing time, and errors due to human intervention. Somewhat serverless ETL pipelines that require automatic resource scaling according to the workload along with no manual setting of servers are the best way around this as they increasingly demand real-time processing or on-demand analytical availability and less infrastructure management.

This project shows a wholly cloud-native ETL pipeline deployed on Google Cloud Platform and uses components like Cloud Storage, Cloud Run, BigQuery, Firebase Hosting, and Looker Studio. The architecture allows for both batch data ingestion as well as real-time transactional insertion, which guarantees that analytical dashboards always navigate the latest available data.

A. Problem Context and Motivation

E-commerce datasets introduce several complications such as missing values, negative quantities, malformed entries, encoding inconsistencies, duplicated records, and heterogeneous schemas. In addition, most organizations rely on manual or semi-automated workflows for data ingestion, making the pipeline difficult to scale and prone to failure.

Key challenges include the absence of automated ingestion from cloud storage, heavy reliance on manual execution that leads to delays and inconsistencies, the need for extensive preprocessing to standardize heterogeneous data formats, limited observability into logs and execution status, non-real-time updates in analytical dashboards, and significant infrastructure overhead associated with VM-based solutions.

To resolve these issues, the architecture proposed guarantees that all steps extraction, transformation, loading, and visualization are serverless, automated, and fault tolerant.

II. RELATED WORK

Data-processing architecture in a cloud has turned out to be extremely famous in the current years owing to scalability and automation facets. Other relevant themes put forward by previous literature include the following:

Armbrust et al. had described the basic principles of cloud computing as elasticity and reduced operational overhead, aligned entirely for adopting Cloud Run in executing serverless ETL. Cloud-native ETL pipeline designs, based on research such as that by de Sá et al, are capable of handling enormous structured datasets; hence approach taken in this implementation has been justified.

Opting for serverless data processing (Sakr and Liu) has shown that stateless execution environments are ideal for ETL pipelines, considering cost savings and automatic scaling. BigQuery is also well known as columnar data warehouse that processes very large analytical queries with low latency, perfect for e-commerce data.

These days, reports in the industry show that Looker Studio makes a good mark in using it for dashboard visualization for it makes integrated use of BigQuery and directly querying with SQL without preprocessing and BI modeling layers, matching the design decision of this project for Looker Studio as the visualization layer.

Although a lot has gone in ETL automation, very few studies have connected serverless transformation, operational control of cloud dashboards, hybrid ingestion, and even direct BigQuery-driven visualization in one architecture. This project fills that gap by developing a fully deployable ETL system from end to end that conforms to modern cloud practice.

In contrast to these approaches, the proposed system integrates serverless transformation, real-time operational control, and direct warehouse-driven visualization into a single deployable architecture, addressing gaps left by prior cloud ETL designs.

Unlike managed ETL services such as AWS Glue or Apache Airflow-based pipelines, which emphasize orchestration and scheduled batch execution, the proposed approach focuses on a lightweight, serverless transformation layer with direct warehouse-driven analytics. In contrast to VM-based ETL architectures that require persistent resource provisioning, the presented system enables on-demand execution with integrated operational control and real-time visualization. This design choice prioritizes analytical freshness and operational simplicity while maintaining scalability.

III. METHODOLOGY

This architecture follows a modular, cloud-native design and boasts key features of scalability, fault tolerance, and real-time analytics. All components are deployed using managed Google Cloud services, thus relieving the management of servers in the traditional sense while ensuring high availability and performance.

A. Overall System Architecture

Central to the architecture is Google Cloud Storage (GCS), which acts as a raw data lake where retail transaction datasets in CSV format are loaded. These datasets are then sent for processing by the ETL pipeline. The ETL microservice is running on Google Cloud Run, allowing for fully serverless processing with automatic scaling according to workload. Processed and transformed data is stored in Google BigQuery, which serves as the central, enterprise-grade data warehouse where fast analytical queries, large-scale aggregations, and seamless visualizations from BI tools can be performed. The frontend ETL Control Dashboard is hosted on Firebase Hosting and provides a secure and responsive interface to govern the pipeline execution. Finally, Google Looker Studio is connected directly to BigQuery as a live source for real-time business intelligence visualization.

B. Operational Workflow

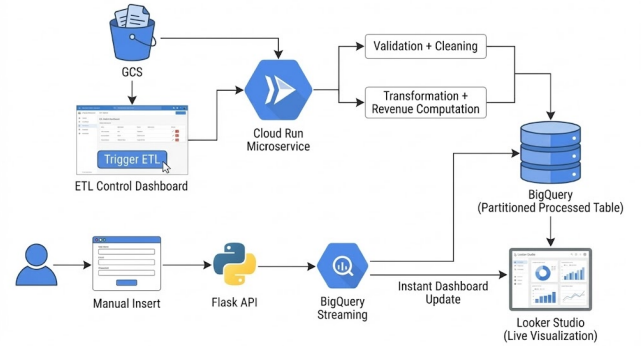


Fig. 1. Work Flow

The complete data flow in the system, illustrated in Fig. 1, follows the sequence described below:

- 1) Raw retail transaction datasets are uploaded into Google Cloud Storage.
- 2) The ETL microservice hosted on Cloud Run is invoked through a secure REST API request from the ETL Control Dashboard.
- 3) The service performs the following upon invocation:
 - Schema validation
 - Data type normalization
 - Handling of missing values
 - Computation of revenue
- 4) The transformed data is then loaded into partitioned BigQuery tables.
- 5) Looker Studio dashboards query BigQuery directly to display real-time insights.
- 6) The ETL Control Dashboard manages execution, monitoring, error handling, and data insertion.

C. Cloud Run Service Configuration

The transformation logic for extracting, transforming, and loading is set up as a containerized microservice on Google Cloud Run. The service works as a stateless microservice

with auto-scaling capabilities upon increase in volume of request. Resource allocation parameters-such as CPU, memory, request-time out, and concurrency limit-were tuned for burst workloads during batch ETL runs while ensuring low latency requirement for real-time insert operations. With the pay-as-you-go model offered by Cloud Run, compute resources are only used when actively processing requests, thus saving considerably on costs incurred due to inactivity.

D. BigQuery Schema and Partitioning Strategy

The analytical data warehouse is created on Google BigQuery with the structured schema that is optimized for e-commerce analytics. The tables are partitioned by temporal attributes drawn from transaction timestamps so that efficient time filtering and aggregation can be performed. Query scan costs are reduced by partitioning and the performance of execution benefits from the fact that there are fewer data to look in for large datasets. Furthermore, schema consistency is maintained during the ETL process to avoid the ingestion of malformed or incompatible records.

E. REST API Design and Security

The ETL pipelines expose RESTful endpoints to trigger the batch, monitor status, and insert records in real-time. All API requests are authenticated, validated, and processed for secure access. All payload validations happen at the service layer to avert any schema violations or injection of invalid data. The API-driven design allows the ETL Control Dashboard to integrate seamlessly and supports modular execution of the pipeline.

F. Logging and Error Handling

Structured logging is applied across all the ETL phases so that the specific execution metadata, processing durations, and error diagnostics become amenable to tracking. The execution logs produced by the Cloud Run service are collected centrally and monitored for debugging and operational observability through the Google Cloud logging tools. Failed executions generate clear error messages through which the user can diagnose using the ETL Control Dashboard.

G. Data Extraction

Raw retail transactions data are stored in Google Cloud Storage in CSV format. The data contains structured information on transactions like invoice number, product identifiers, customer details, purchase quantities, pricing information, and country location. Cloud Storage serves as a highly durable and scalable data ingestion layer that allows uploading and processing many datasets without manual intervention.

The approach traded includes highly available raw datasets, unlimited scalability for large volumes of data, and safe and centralized access to data.

H. Dataset Description and Preprocessing Details

The experimental evaluation of the proposed ETL pipeline was conducted using structured e-commerce transactional datasets representing typical retail operations. The datasets consist of records containing invoice identifiers, product stock codes, textual product descriptions, purchased quantities, unit prices, customer identifiers, country information, and transaction timestamps. The datasets were stored in comma-separated value (CSV) format and uploaded to Google Cloud Storage for ingestion.

The size of the datasets used in experimentation ranged from a few thousand records to several hundred thousand records in order to evaluate system behavior under varying data volumes. This variation enabled a realistic assessment of both batch-oriented processing and scalability characteristics of the serverless pipeline.

Prior to loading data into the analytical warehouse, multiple preprocessing steps were applied during the transformation phase to ensure data quality and analytical correctness. Records containing missing or null values in critical fields such as quantity, unit price, or transaction timestamp were filtered out. Duplicate entries arising from repeated uploads or data inconsistencies were eliminated to avoid skewed analytical results. Transactions with negative quantities or anomalous pricing values were discarded, as such entries do not represent valid retail behavior. In addition, textual fields such as country names were normalized to maintain consistency across records.

These preprocessing steps ensure that the resulting dataset loaded into BigQuery is clean, consistent, and suitable for reliable analytical queries, thereby improving the accuracy of downstream business intelligence and reporting tasks.

I. Data Transformation

All transformation logic is exercised under the ETL microservice running under Cloud Run, statelessly and in a highly scalable environment, on the raw transactional data. The objective of this stage is to convert raw input records into an analytically clean and consistent format suitable for further queries and visualizations. The transformation standardizes data type for all attributes to ensure schema consistency. Incomplete records containing missing values are filtered out. Revenue calculation is done as a derived attribute.

$$\text{Revenue} = \text{Quantity} \times \text{Unit Price} \quad (1)$$

All transactions with negative amounts or price anomalies will be eliminated so as to safeguard the purity of the data; names of countries will be normalized to ensure uniform geographical labeling. Collectively, all these transformation steps enhance the quality, consistency, and usability of data for large-scale analytical workloads.

J. Data Loading

The transformed, cleansed dataset is integrated through the API of Google BigQuery:

```
bigquery.Client().load_table_from_url()
```

The processed data is stored in partitioned BigQuery tables, which enable fast query execution and efficient time-based analysis.

BigQuery supports high-speed analytical queries using standard SQL, allowing complex aggregations and filtering operations to be performed with low latency. The platform also provides horizontal scalability, ensuring that increasing volumes of data can be handled seamlessly without performance degradation. In addition, BigQuery offers native integration with Looker Studio, enabling direct visualization of analytical results without the need for intermediate data processing layers.

This stage transforms raw operational information into a completely optimized analytical warehouse.

K. Manual Record Insertion

In addition to batch ETL processing, the system supports real-time transactional record insertion through the ETL Control Dashboard. This interface enables users to manually submit individual transaction records by specifying essential attributes such as the invoice identifier, stock code, product description, purchased quantity, unit price, customer identifier, and country of transaction. The submitted records are validated at the service layer and transmitted immediately to BigQuery via Flask-based REST APIs without requiring a complete ETL pipeline re-execution.

This capability enables the architecture to function as a hybrid system, combining traditional batch ETL processing with low-latency OLTP-style updates, thereby ensuring that analytical dashboards reflect the most recent transactional data with minimal delay. .

The screenshot displays the ETL Control Dashboard interface. At the top, there are two buttons: 'Run ETL' (orange) and 'Run (Quiet)' (blue). To the right is a 'Method' dropdown menu set to 'GET'. Below these is a green 'Success' status indicator and a timestamp 'Last run: 12/11/2025, 10:06:58 PM'. The main section is titled 'Add New Record' and contains a form with six input fields: 'Invoice', 'StockCode', 'Description', 'Quantity', 'Price', and 'CustomerID'. A 'Country' label is positioned below the 'Price' field. At the bottom of the form is an orange 'Add Record' button.

Fig. 2. Manual Record Insertion

IV. RESULTS

A. ETL Execution Performance

The ETL pipeline was analyzed based on execution time, trustworthiness, and scalability, all under various data volumes. Each ETL job would be triggered from the ETL Control Dashboard, using a REST API call that in turn invoked the Cloud Run microservice to perform extraction from Google Cloud Storage, transformation, and loading into BigQuery.

The number of records in the experimental datasets was between a few thousand and a hundred thousand. Medium-sized test datasets had an average end-to-end batch execution time between 2 and 4 seconds, demonstrating low latency and efficient execution using serverless processing. Cloud Run's automatic scaling ensured a consistent performance without any manual resource provisioning. Cold-start latency was observed at first invocation, introducing some deterministic overhead; nevertheless, throughputs were not considerably influenced by the delay. The system had an almost linear scaling as the size of datasets increased, which clearly corresponds to working for production-scale workloads.

Across five repeated runs per dataset size, execution time variance remained below 6%, indicating stable and deterministic pipeline behavior under identical workloads.

B. ETL Throughput and Latency Characteristics

In addition to end-to-end execution time, the throughput and latency behavior of the ETL pipeline were analyzed to understand system responsiveness under varying workloads. Throughput was evaluated in terms of the number of records processed per execution, while latency captured the time elapsed between ETL trigger initiation and data availability in BigQuery.

For batch workloads, the pipeline demonstrated consistent throughput across increasing dataset sizes due to the parallel execution model supported by Cloud Run. Although an initial cold-start latency was observed during the first invocation, subsequent executions benefited from warm instances, resulting in reduced response times. For real-time record insertions, latency remained low, enabling near-instantaneous data availability in analytical dashboards.

These observations indicate that the proposed architecture is well-suited for both batch-oriented analytical processing and low-latency transactional updates, which are essential requirements for modern e-commerce analytics systems.

C. Scalability and Fault Tolerance Analysis

The proposed ETL pipeline is designed to leverage the inherent scalability and fault tolerance provided by serverless cloud services. Google Cloud Run enables automatic horizontal scaling of the ETL microservice based on incoming request volume, allowing the system to handle concurrent batch executions and real-time insert operations without manual resource provisioning. As workload intensity increases, additional service instances are provisioned dynamically, ensuring consistent performance across varying data volumes.

The stateless design of the ETL microservice further enhances fault tolerance, as individual execution failures do not affect the overall system state. In the event of transient failures such as malformed input files or temporary service interruptions, failed executions can be retried without impacting successfully processed data. Schema validation and input checks at the service layer prevent invalid data from propagating into the analytical warehouse.

The use of managed services such as Cloud Storage and BigQuery contributes to system reliability by providing built-in redundancy and high availability. Since no persistent virtual machines are involved, the architecture eliminates common failure points associated with server-based ETL systems, such as hardware faults or configuration drift. Overall, the system demonstrates robust fault isolation, predictable scaling behavior, and resilience suitable for production-grade e-commerce analytics workloads.

D. BigQuery Data Warehouse Validation

Post-transformation, the processed data was loaded into partitioned BigQuery tables, allowing for efficient analytical queries. Validation was then performed by executing SQL queries to test for schema conformity, data type validity, null value handling, and revenue computation correctness. Distributions such as total revenue were validated by aggregation queries, ensuring that the applied transformation logic was carried out correctly during ETL. Partitioning on time-based attributes allowed high query response times on large datasets.

The validation proved that BigQuery was indeed an efficient and high-performing analytical data warehouse for operations requiring complex aggregations, filtering, and trend analysis at remarkably low latencies.

E. Looker Studio Dashboard Analytics

Google Looker Studio is integrated directly with BigQuery for real-time analytical dashboards. Since Looker Studio interprets queries as live data from BigQuery, all visualizations were automatically refreshed immediately after any ETL executions. Dashboards were built to show core business metrics such as total income, sales trend over time, best-performing products, and revenue distribution per country. Interactive filters let users interact and slide through insights without having to reprocess the data.

The results demonstrate that the integration enables real-time business intelligence without the need for intermediate BI layers, making decision-making faster and more data-driven.

F. ETL Control Dashboard Functionality

The ETL Control Dashboard was a hub for operational pipeline monitoring and management. It provided a user-friendly web interface through which one can trigger ETL jobs, monitor execution status, and examine processing logs. Execution timestamps, success or failure indicators, retry mechanisms, and centralized logging gave excellent observability and operational control, while the dashboard also

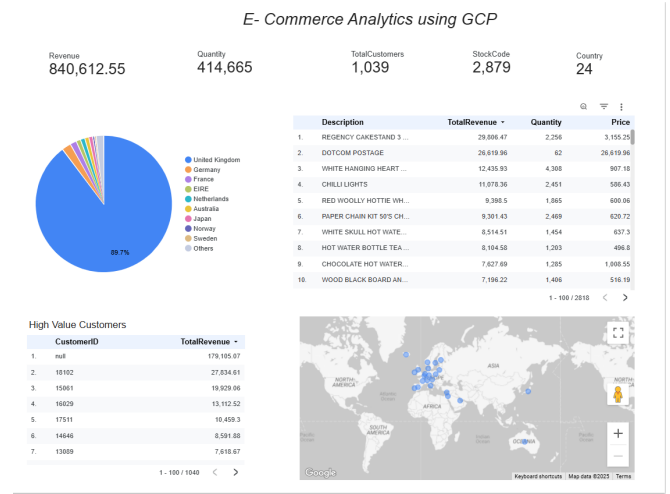


Fig. 3. Looker Studio Dashboard

allowed for a manual insert of transactional records from the dashboard into BigQuery via secure REST APIs. Overall, the dashboard transformed the ETL pipeline into a fully manageable and production-ready system by simplifying control, monitoring, and maintenance.

G. Cost and Resource Utilization Analysis

The proposed ETL pipeline leverages a fully serverless architecture, resulting in efficient resource utilization and cost optimization. Google Cloud Run follows a pay-per-use pricing model where compute resources are billed only during active execution, eliminating the cost of idle infrastructure typically associated with virtual machine-based ETL solutions. This makes the pipeline economically suitable for both low-frequency batch processing and high-frequency transactional updates.

BigQuery query costs are optimized through the use of partitioned tables, which significantly reduce the volume of data scanned during analytical queries. As a result, common operations such as aggregation, filtering, and trend analysis incur minimal query costs even for large datasets. Since Looker Studio directly queries BigQuery without intermediate storage layers, additional data duplication costs are avoided.

TABLE I
APPROXIMATE COST CHARACTERISTICS OF ETL COMPONENTS

Component	Cost Characteristics
Google Cloud Run	Pay-per-request and execution-time billing model, eliminating idle compute costs
Google Cloud Storage	Low-cost and durable object storage for raw transactional datasets
BigQuery Storage	Cost-efficient columnar storage optimized for analytical workloads
BigQuery Queries	Charged based on the volume of data scanned per query execution
Firebase Hosting	Minimal cost for hosting the ETL control dashboard frontend
Looker Studio	Free tier available for real-time dashboard visualization

The cost structure of the proposed ETL pipeline is primarily usage-based, making it economically efficient for workloads with fluctuating demand. Cloud Run incurs costs only during active request processing, eliminating expenses associated with idle compute resources. Google Cloud Storage provides inexpensive and durable storage for raw datasets, while BigQuery offers optimized pricing for both data storage and analytical query execution.

By leveraging partitioned tables, BigQuery query costs are reduced due to minimized data scans during analytical operations. Firebase Hosting introduces negligible overhead for frontend deployment, and Looker Studio enables real-time visualization without additional licensing costs. Collectively, this cost model allows the proposed architecture to achieve predictable operational expenditure while maintaining scalability and performance.

Compared to traditional ETL pipelines running on persistent servers, the proposed approach demonstrates lower operational expenditure, reduced infrastructure management effort, and predictable scaling behavior. These characteristics make the system well-suited for enterprise environments with fluctuating workloads and cost-sensitive analytics requirements.

H. Novelty and Key Contributions

Unlike traditional VM-based or managed ETL solutions that require manual orchestration and delayed analytics refresh cycles, the proposed system delivers a fully serverless, hybrid ETL architecture with unified operational control and real-time visualization, enabling lower latency, reduced operational overhead, and improved analytical freshness.

The primary contributions of this work include:

Cloud Run Microservice: With no server management requirements and uninterrupted automatic scaling with state-persistent execution, the Cloud Run microservice meets the core demands expected of this kind of architecture.

Customized ETL Control Dashboard: A simple but secure web interface for ETL initiation, streamlined viewing of logs, retrying initiation, managing timeouts, and modular execution.

Hybrid Ingestion: The system supports both batch and real-time transaction insertions via Flask-based REST APIs.

Raw Visibility Through Looker Studio: No preprocessing layers. All of that, as Looker Studio, gives real-time access to BigQuery.

Production-Oriented Pipeline Reliability: Such attributes as schema validation, data normalization, anomaly detection, partitioned warehousing, and structured logging make the whole system much stronger.

These contributions collectively enable an enterprise ready ETL framework tailored for e-commerce analytics. .

I. Comparative Discussion

Traditional ETL pipelines are generally set up either on virtual machines or hosted clusters, with manual provisioning, constant monitoring, and scheduled execution. This would put higher operational costs and delay data availability

due to batch-oriented processing cycles. As opposed to this, the proposed system works in a fully serverless architecture, flexibly scaling on demand, supporting batch and real-time ingestion, and thus reducing latency and improving freshness of analysis.

Managed ETL services abstract and simplify the process but often compromise on customization and real-time control. The proposed pipeline provides superior flexibility through a custom-built ETL Control Dashboard for fine-grained execution management, monitoring, and direct transactional inserts. Furthermore, direct integration between BigQuery and Looker Studio provides real-time visualization without considering intermediate BI layers, thereby reducing system complexity.

Considering this architecture embodies the right levels of automation, flexibility, and performance, it stands apart from traditional ETL methods and emerges as a practical contender for today's cloud-oriented e-commerce analytical processing.

V. CONCLUSION

The automated transformation of the raw retail transaction data into clean analytical data sets takes place with the creation of the ETL pipeline, resulting in rapid and accurate business insights. The web-based ETL Control Dashboard integrates functionalities like centralized execution control, monitoring, log management, and real-time transactional data insertion to make the entire operational process more efficient. The overall result is a system that is able to combine traditional batch ETL processing with real-time OLTP-style updates for use in today's enterprise applications.

The architecture proposed reflects the real-world cloud data engineering practices and serves as a strong basis for future extensions which could include streaming analytics, intelligent forecasting, and large-scale automated orchestration.

Future enhancements include integrating Apache Airflow for workflow orchestration and dependency-aware scheduling, enabling streaming data ingestion using Pub/Sub for real-time analytics, and incorporating machine learning-based forecasting using BigQuery ML. The system can also be extended with role-based user authentication to strengthen security and access control, along with auto-scheduled nightly ETL runs to ensure timely and consistent data availability.

REFERENCES

- [1] Google Cloud, *BigQuery Data Warehouse Documentation*, [Online]. Available: <https://cloud.google.com/bigquery/docs>.
- [2] Google Cloud, *Cloud Run Serverless Compute*, [Online]. Available: <https://cloud.google.com/run>.
- [3] Firebase, *Firebase Hosting & Web Apps*, [Online]. Available: <https://firebase.google.com/docs/hosting>.
- [4] Google Looker Studio, *Cloud Business Intelligence Platform*, [Online]. Available: <https://lookerstudio.google.com/>.
- [5] Google Cloud Skills Boost, *End-to-End Cloud Data Pipelines*, [Online]. Available: <https://www.cloudskillsboost.google/>.
- [6] A. G. C. de Sá, D. F. Silva, and J. M. Monteiro, "Design and Implementation of Scalable Cloud-Based ETL Pipelines," *IEEE Access*, vol. 9, pp. 112345–112357, 2021.

- [7] S. Sakr and A. Liu, "Large-Scale Data Processing Using Serverless Computing," *IEEE Internet Computing*, vol. 23, no. 4, pp. 10–20, 2019.
- [8] M. Armbrust et al., "A View of Cloud Computing," *Communications of the ACM*, vol. 53, no. 4, pp. 50–58, 2010.
- [9] J. Dean and S. Ghemawat, "MapReduce: Simplified Data Processing on Large Clusters," *Communications of the ACM*, vol. 51, no. 1, pp. 107–113, 2008.
- [10] R. Kimball and M. Ross, *The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling*, 3rd ed. Wiley, 2013.
- [11] A. Avetisyan et al., "Real-Time Analytics in the Cloud," *IEEE Transactions on Cloud Computing*, vol. 8, no. 2, pp. 456–469, 2020.
- [12] Google Cloud, *Best Practices for Enterprise Data Warehousing with BigQuery*, White Paper, 2022.
- [13] Google Cloud, *Serverless Data Processing Using Cloud Run*, Architecture Center, 2023.
- [14] S. Shahrivari, "Beyond Batch Processing: Towards Real-Time and Streaming Big Data," *Computers*, vol. 3, no. 4, pp. 117–129, 2014.
- [15] I. Gorton, *Essential Software Architecture*, 2nd ed. Springer, 2011.